

ARCHITECTURE REFERENCE

Workout Tracker

How a strength-training log is put together — from a tap on a React screen to a row in SQLite, and back up again. Four views of the same system: how a request moves, how a session is established, where the code lives, and how the data is shaped.

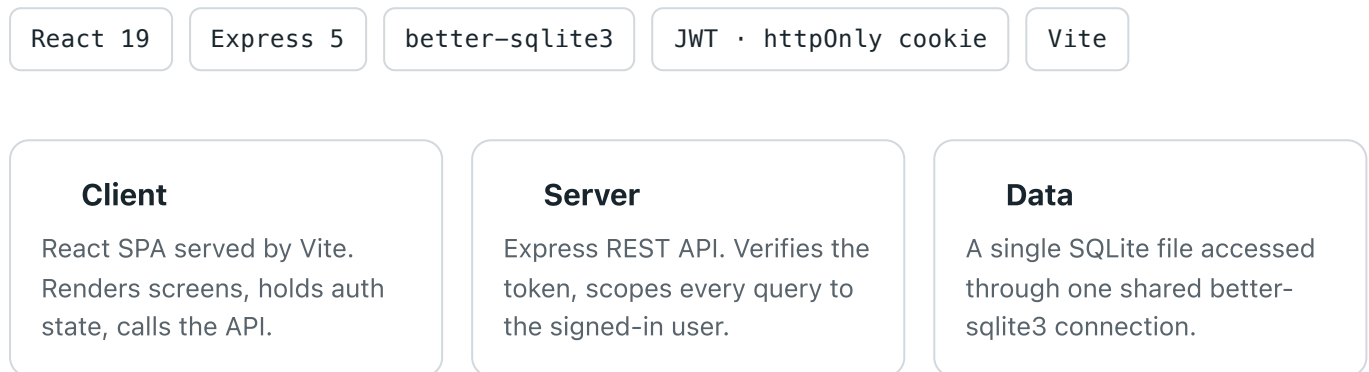


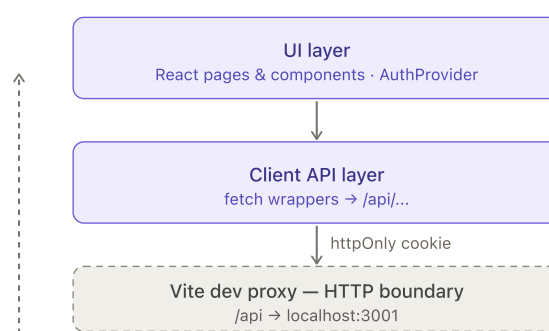
FIG 1 — RUNTIME

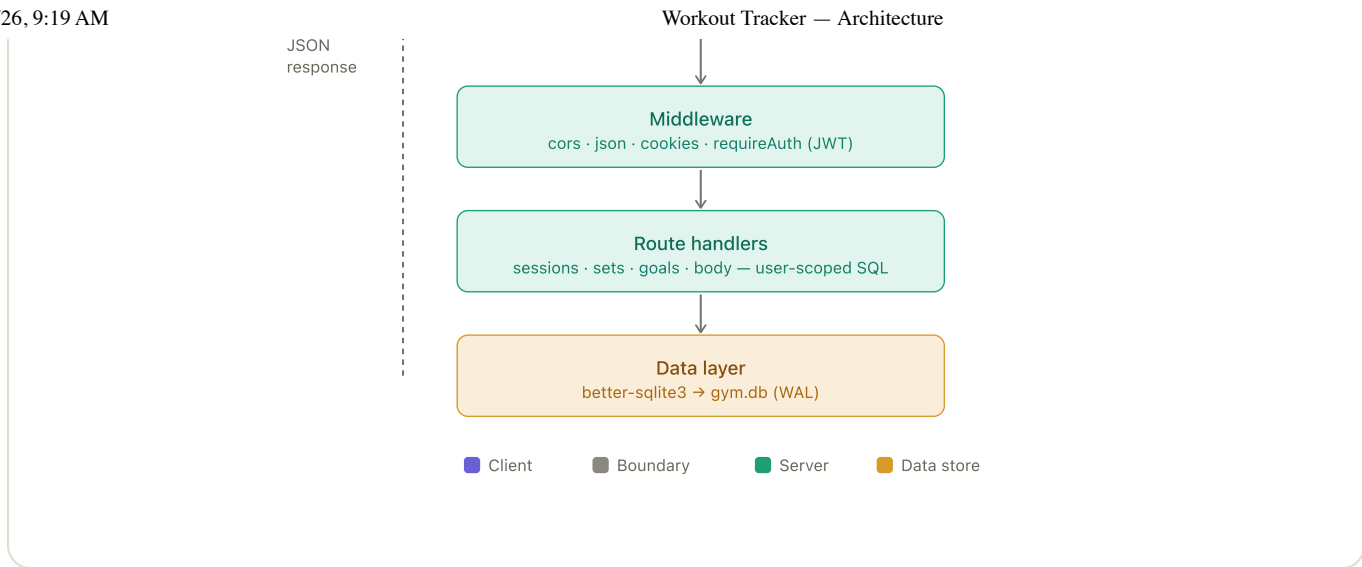
A request moves down; the response comes back up

Two tiers talk over HTTP. In development a Vite proxy forwards `/api` to the API on port 3001; in production the two are served same-origin, so the proxy collapses but the `/api` path contract is unchanged. Every non-auth route passes through `requireAuth`, which reads the JWT from the `httpOnly` cookie, verifies it, and attaches `req.user` — so handlers can trust who is asking before they touch the database.

FIG. 01

LAYERED REQUEST FLOW



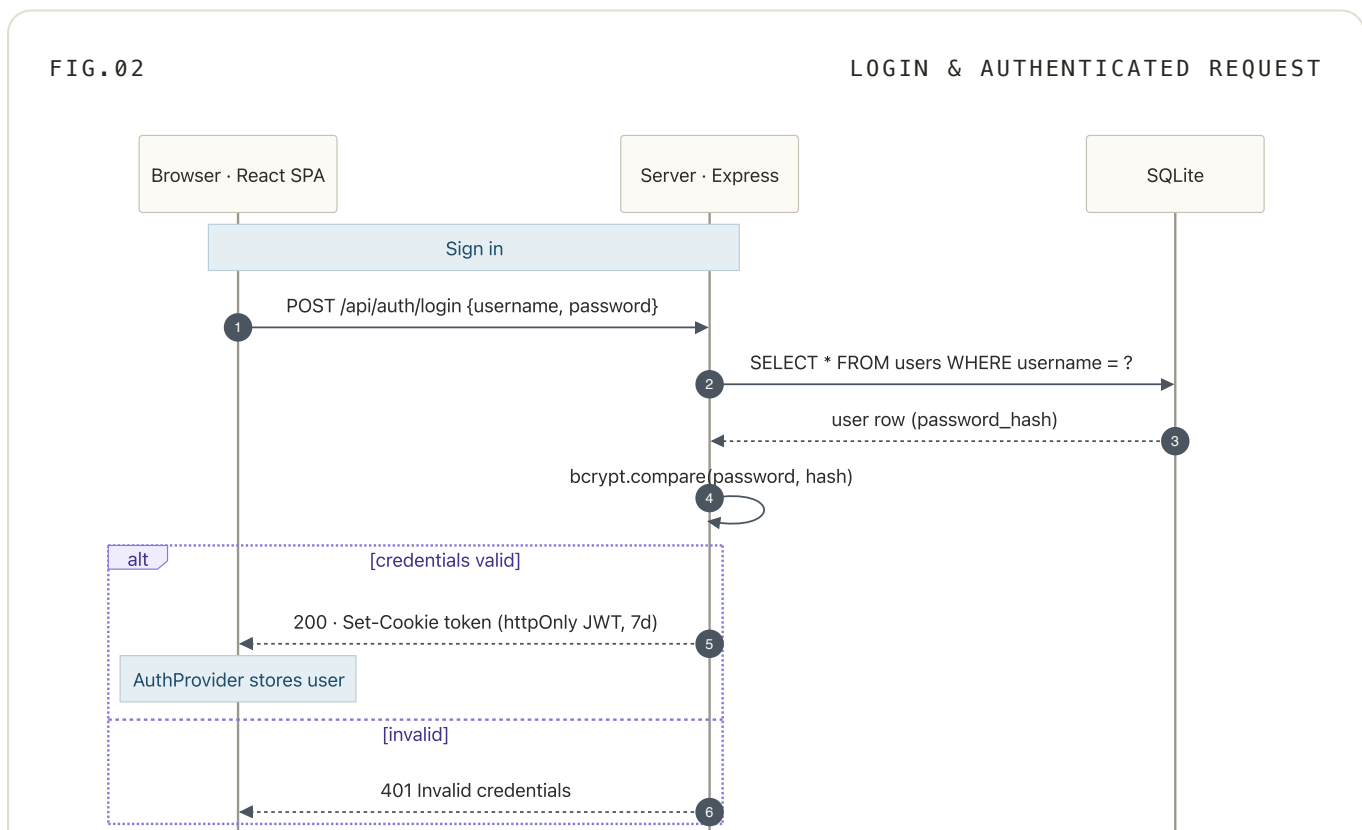


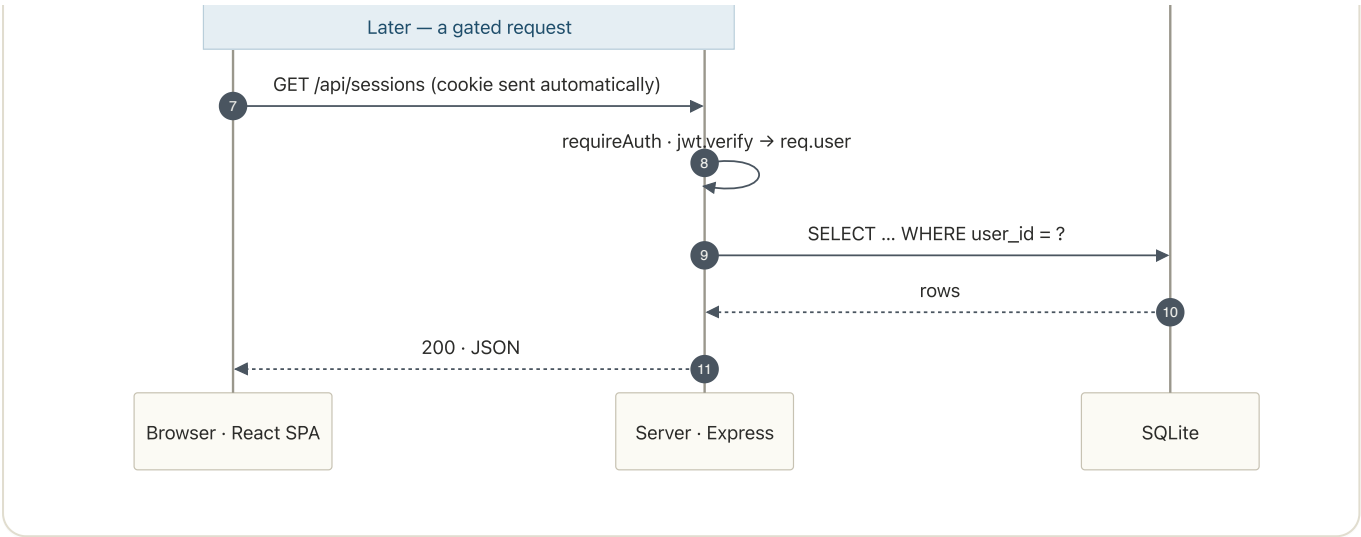
Two mirrored gates. `ProtectedRoute` guards the UI, but that is only UX — `requireAuth` on the server is the real enforcement, and every personal-data query additionally filters on `user_id`.

FIG 2 — AUTHENTICATION

How a session is established, then used

Auth is a signed JWT carried in an `httpOnly` cookie. Logging in verifies the password with `bcrypt` (cost 12) and sets the cookie; from then on the browser attaches it to every call automatically. On each gated route `requireAuth` verifies the token, sets `req.user`, and the handler scopes its SQL to that user. The token is stateless and lasts 7 days — `logout` clears the cookie but can't revoke a token that was already issued.



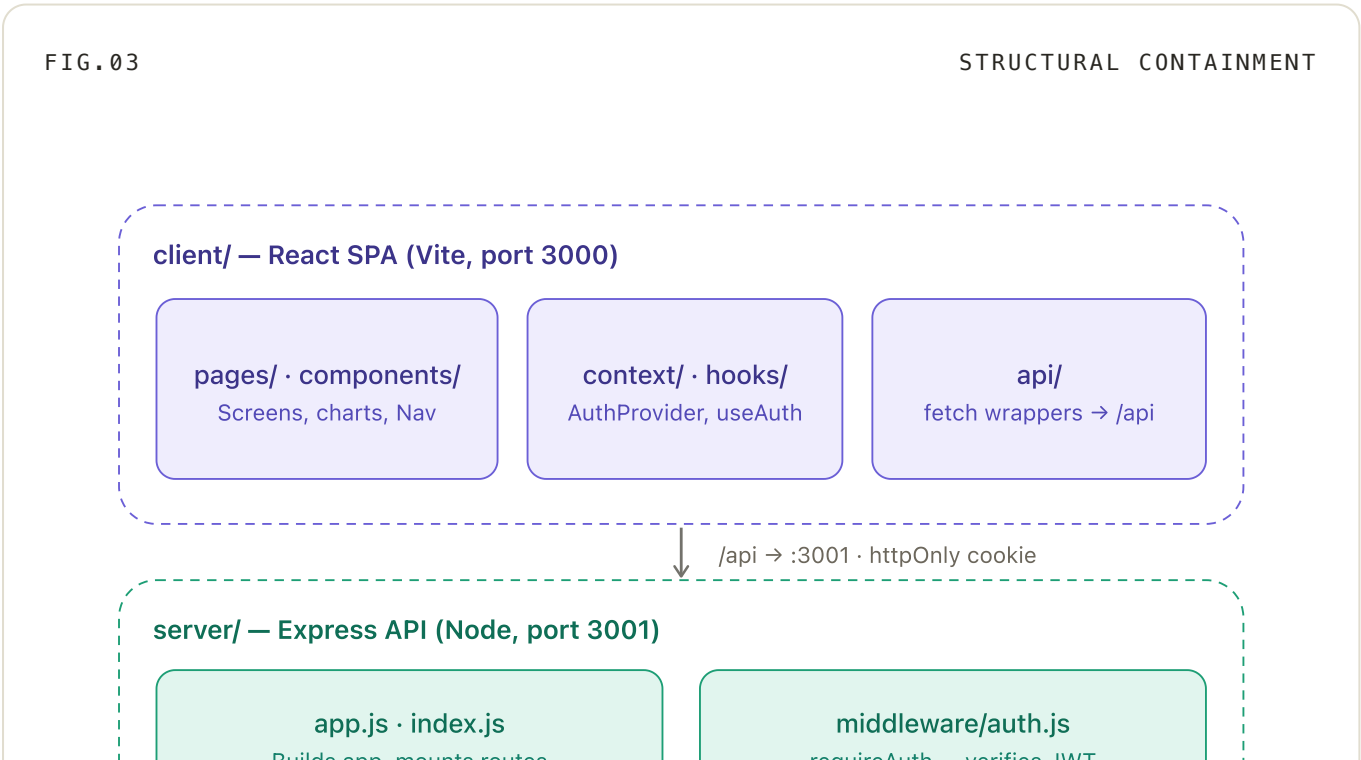


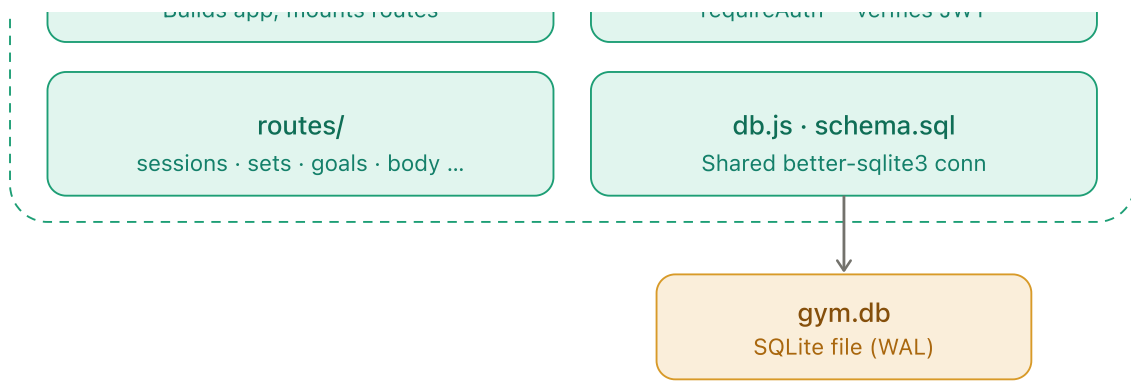
The cookie is never read by JS. Being `httpOnly`, the SPA can't see the token — it relies on the browser attaching it, and calls `GET /api/auth/me` on load to rehydrate the session.

FIG 3 — SOURCE LAYOUT

The same system, organised by where the code lives

The repository is one workspace with two halves. `client/` is the React SPA — screens and charts, the auth context and its hook, and a thin `api/` layer of `fetch` wrappers that is the single seam between UI and backend. `server/` is the Express API — the app wiring, the auth middleware, the route handlers, and one data-access module. Everything below the API talks to a single SQLite file.





One exception to the seam. `AuthProvider` calls `/api/auth/*` directly rather than through `api/`, because it bootstraps auth state before the rest of the app renders.

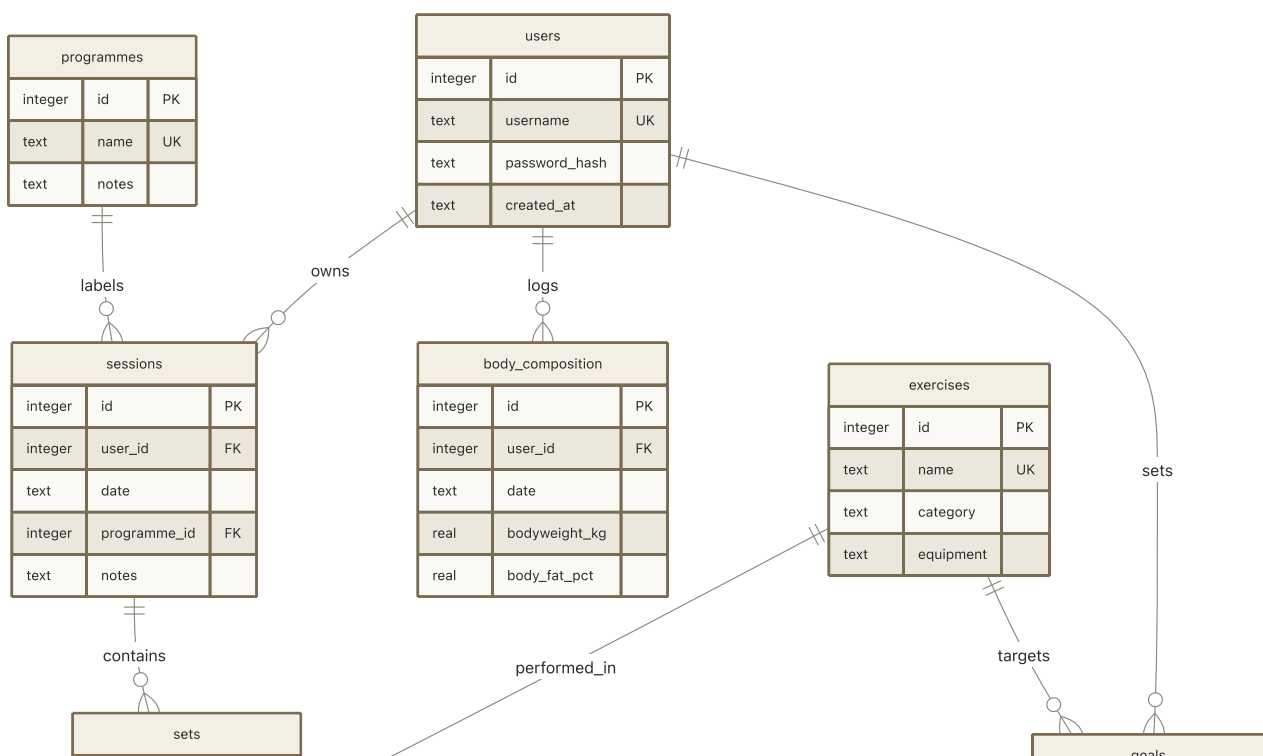
FIG 4 — PERSISTENCE

The data model behind the log

Seven tables. `users` owns the personal data — `sessions`, `goals` and `body_composition` each carry a `user_id`. A `session` holds many `sets`; each set references a row in the shared `exercises` catalogue. `programmes` optionally label a session. Notably, `sets` have no `user_id` of their own — they inherit ownership through their parent session.

FIG. 04

SQLITE SCHEMA — GYM.DB



integer	id	PK
integer	session_id	FK
integer	exercise_id	FK
integer	set_number	
real	weight_kg	
integer	reps	



integer	id	PK
integer	user_id	FK
integer	exercise_id	FK
text	target_type	
real	target_value	

Simplified — a few sparse columns on **sets** (`weight_note`, `duration_min`, `rest_min`, ladder fields) are omitted for legibility.

OWNERSHIP

`sessions`, `goals` and `body_composition` carry `user_id` directly; `sets` inherit it through their session, so set queries join back to the session to enforce it.

DELETES

- `sets` → `sessions` cascades (delete a session, its sets go too).
- `sets` → `exercises` restricts (can't delete a used exercise).
- `sessions` → `programmes` sets null (deleting a programme just unlabels its sessions).

SHARED TABLES

`exercises` and `programmes` are reference tables, not per-user — every account draws from the same catalogue.

UNIQUENESS

`body_composition` has `UNIQUE(user_id, date)` — one measurement per person per day.

REGISTRATION

Open registration is locked to the first user; further accounts are created through the authenticated `/api/auth/create-user` route.

repo [Workout Tracker](#) branch `multi-user-accounts` stack `React · Express · SQLite`